BZ205 Konu Anlatımı

Listeler ve Bağlantılı Listeler

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

4 Listeler ve Bağlantılı Listeler

4.1 Veri yapısı ve soyut veri tipi

Bir veri yapısı tanımlarken iki şey belirlenir: (1) **nesneler kümesi** (ne depolanacak: tam sayı, ondalık, karakter, öğrenci kaydı…), (2) **işlemler kümesi** (bu veriler üzerinde hangi işlemler yapılacak). **Soyut veri tipi (SVT) = soyut veri yapısı**: dildeki temel tipleri kullanarak oluşturduğumuz yeni tip (kum + çimento → beton benzetmesi). C++'ta `class` ile tanımlanır.

- **class – struct farkı:** class içinde değişkenler *ve* fonksiyonlar tanımlanabilir (fonksiyonu sadece sınıf içindekiler kullanır); struct'ta fonksiyonlar dışarıda tanımlanır ve başka yerlerden erişilebilir (risk).
- **SVT avantajları:** modülerlik (kütüphane bir kez yazılır), bilgi gizleme, hızlı prototipleme. **Dezavantajı:** verimlilik kaybı (kullanılmayan yetenekler de yüklenir).

4.2 Dizi ile bağlantılı liste karşılaştırması

	Dizi	Bağlantılı liste
Boyut	Sabit, derleme anında verilir	Değişken (bellek elverdiği sürece)
Bellek	Ardışık *blok* hafıza gerekir (6'lık dizi 5'lik boşluklara sığmaz)	Düğümler bellekte dağınık olabilir, bağlantılarla birleşir
Veri tipi	Tek tip, hücrede tek veri (öğrenci için 3 ayrı dizi: no, ad, soyad)	Düğümde birden çok/farklı tipte veri (no, ad, soyad birlikte)
Başa ekleme/silme	Tüm elemanlar kayar: $O(n)$	Sabit: $O(1)$
k. eleman (findKth)	Sabit: $O(1)$	Baştan gezilir: $O(n)$
Bulma, yazdırma	$O(n)$	$O(n)$

4.3 Liste işlemleri (slayt örneği: 34, 12, 52, 16, 12)

- `printList`: listeyi yaz. `makeEmpty`: listeyi boşalt (dönem bitti, yeni ders).
- `find(52)` → 3 (konumu döner; bulunamazsa 0/NULL). Sıralı olmadığı için baştan tek tek bakılır.
- `insert(3, x)` → 34, 12, 52, **x**, 16, 12 (3. veriden sonra ekle; tren vagonu araya eklemek gibi).
- `remove(52)` → 34, 12, 16, 12 (vagonu çıkarıp önceki ve sonrakini bağlamak).
- `findKth(5)` → 12 (5. eleman). *Ders varsayımı: her veriden bir tane vardır.*

4.4 Bağlantılı liste gerçekleştirmesi (C++)

```
class Node {
public:
    double data;      // veri (ogrenci icin no, ad, soyad olabilir)
    Node*  next;      // sonraki dugume isaretci
};
class List {
public:
    List(void) { head = NULL; }             // yapici: bos liste
    ~List(void);                            // yikici
    bool  IsEmpty() { return head == NULL; }
    Node* InsertNode(int index, double x);
    int   FindNode(double x);
    int   DeleteNode(double x);
    void  DisplayList(void);
private:
    Node* head;       // ilk dugumu gosterir; verisi yok (lokomotif)
};
```

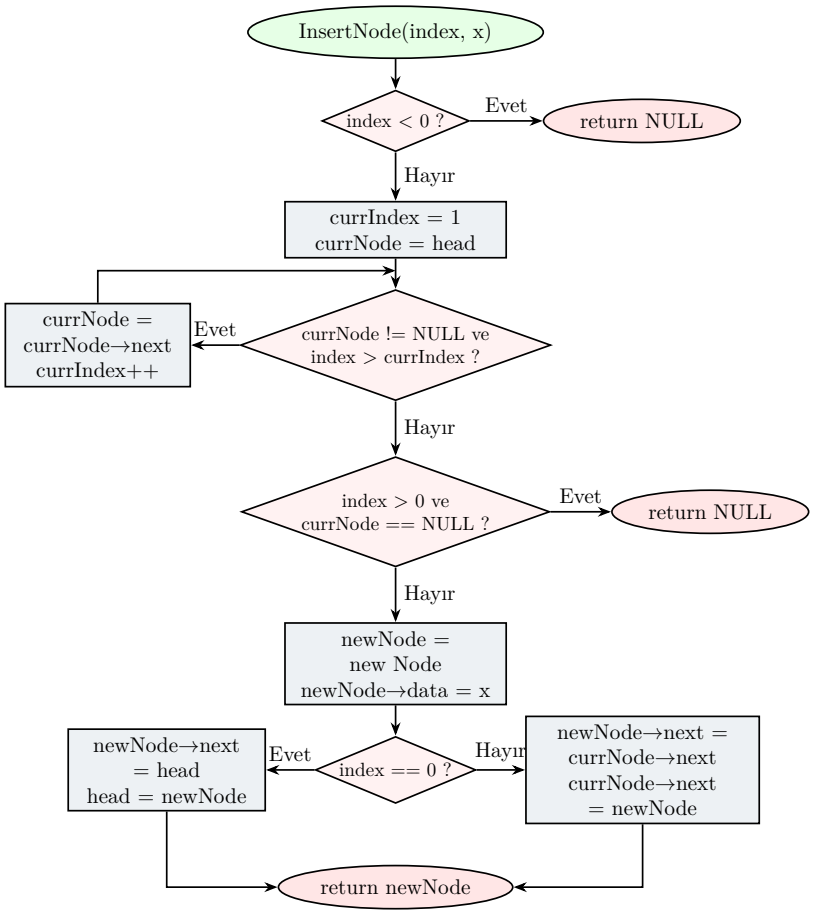
`head` *private*'tır: listenin başlangıcını dışarıdan kimse değiştirememeli. Son düğümün `next`'i NULL'dır.

InsertNode – belirtilen konumdan sonra ekleme

```
Node* List::InsertNode(int index, double x) {
    if (index < 0) return NULL;
    int currIndex = 1;
    Node* currNode = head;
    while (currNode && index > currIndex) {    // index'inci dugumu bul
        currNode = currNode->next;
        currIndex++;
    }
    if (index > 0 && currNode == NULL) return NULL;   // oyle bir dugum yok
    Node* newNode = new Node;
    newNode->data = x;
    if (index == 0) {                          // basa ekle
        newNode->next = head;
        head = newNode;
    } else {                                   // currNode'dan sonra ekle
        newNode->next = currNode->next;
        currNode->next = newNode;
    }
    return newNode;
}
```

Bağlantı sırası

Araya eklemede önce `newNode->next = currNode->next`, *sonra* `currNode->next = newNode` yapılır. Ters sırada yapılırsa listenin geri kalanı kaybolur.



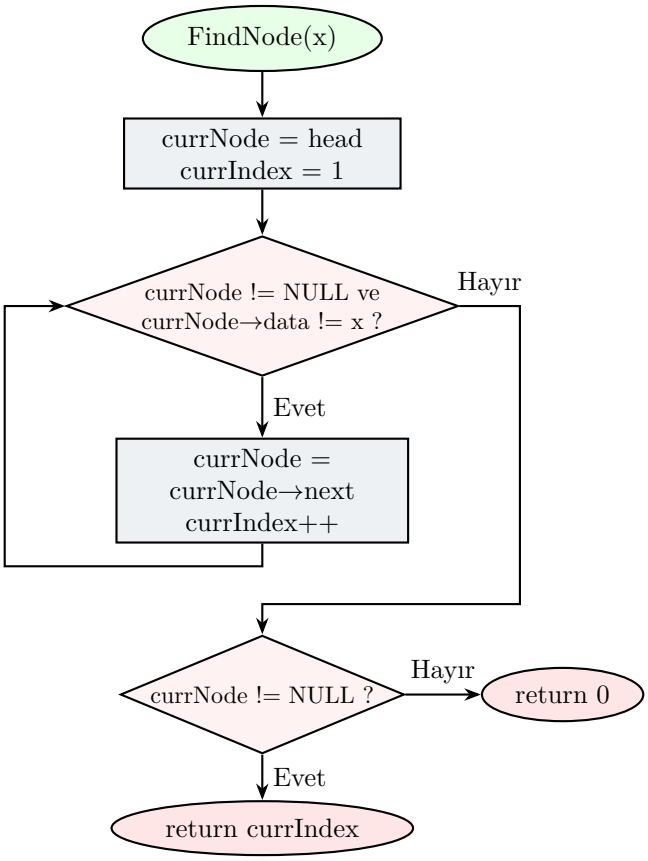
Yürütme zamanı: en kötü (sona ekleme) $T(n) = 3n + 8 \Rightarrow O(n)$; en iyi (başa ekleme) sabit $\Rightarrow \Omega(1)$ (ayrıntı: çözüm dosyası).

FindNode ve DeleteNode

```
int List::FindNode(double x) {
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) return currIndex;
    return 0;                                   // bulunamadi
}
int List::DeleteNode(double x) {
    Node* prevNode = NULL;
    Node* currNode = head;
    int currIndex = 1;
    while (currNode && currNode->data != x) {
        prevNode = currNode;
        currNode = currNode->next;
        currIndex++;
    }
    if (currNode) {
        if (prevNode) prevNode->next = currNode->next;   // aradan cikar
        else          head = currNode->next;             // bastaki silindi
        delete currNode;
        return currIndex;
```

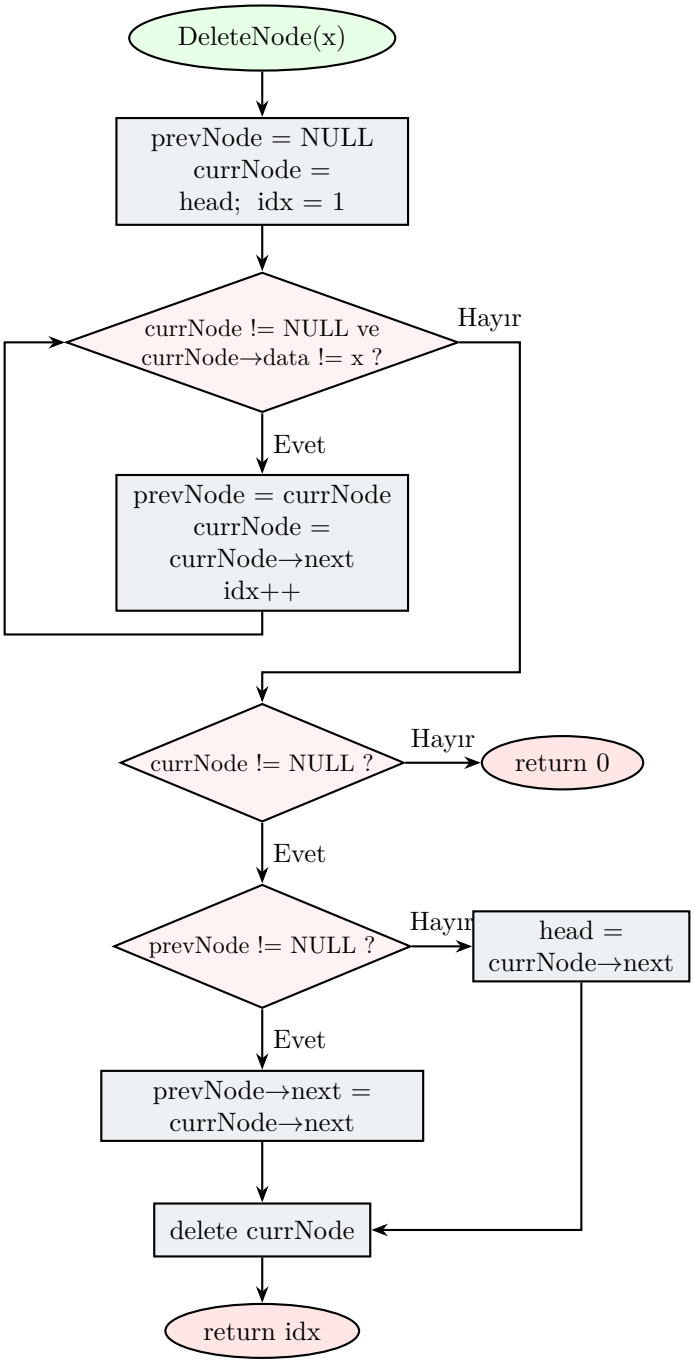
```
    }
    return 0;
}
void List::DisplayList() {
    int num = 0;
    Node* currNode = head;
    while (currNode != NULL) {
        cout << currNode->data << endl;
        currNode = currNode->next;
        num++;
    }
    cout << "Dugum sayisi: " << num << endl;
}
```

Akış diyagramı: FindNode (listede arama)



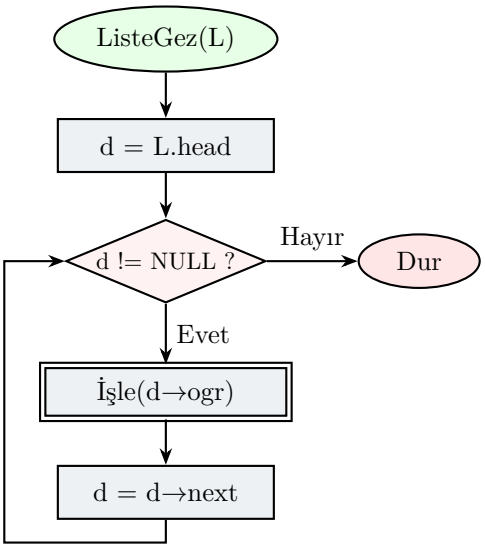
Farz edin ki sorularında listede öğrenci aramanın kalıbıdır.

Akış diyagramı: DeleteNode (listeden silme)



Silinecek düğümün *öncekisi* gerektiği için `prevNode` tutulur; başlaki siliniyorsa `head` güncellenir.

Akış diyagramı: listeyi baştan sona gezme (DisplayList kalıbı)



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Zaman ve alan karmaşıklığı

İşlem	En iyi	En kötü	Ek alan	Not
`InsertNode` (konum k)	$\Omega(1)$	$O(n)$	$O(1)$	en iyi başa ekleme; en kötü sona ekleme, $T(n) = 3n + 8$
`FindNode`	$\Omega(1)$	$O(n)$	$O(1)$	en iyi aranan ilk düğümde; en kötü sonda veya yok
`DeleteNode`	$\Omega(1)$	$O(n)$	$O(1)$	silinecek düğüm önce aranır; bağlantıyı değiştirmek sabit
`DisplayList`	$\Theta(n)$	$\Theta(n)$	$O(1)$	her düğüm bir kez yazılır
`findKth`(k)	$\Omega(1)$	$O(n)$	$O(1)$	dizide $O(1)$; listede baştan k düğüm gezilir

4.5 Liste çeşitleri

- **Dairesel bağlantılı liste:** son düğüm NULL yerine ilk düğümü gösterir.
- **Çift yönlü bağlantılı liste:** her düğümde `next` ve `prev` vardır; geriye doğru gezilebilir, silmede önceki düğümü aramaya gerek kalmaz.

Sınav bağlantısı

Farz edin ki sorularında liste gezme kalıbı: `d = liste.head; while (d != NULL) { …d = d->next; }`. Hocanın kabul ettiği temel işlemler: `head`, `next`, `FindNode/find`, `InsertNode`, `DeleteNode`. Yılı bilinmeyen bir vizede *"bağlantılı liste ekleme akış diyagramı, karmaşıklığı"* soruldu ve fonksiyonu öğrencinin yazması istendi.